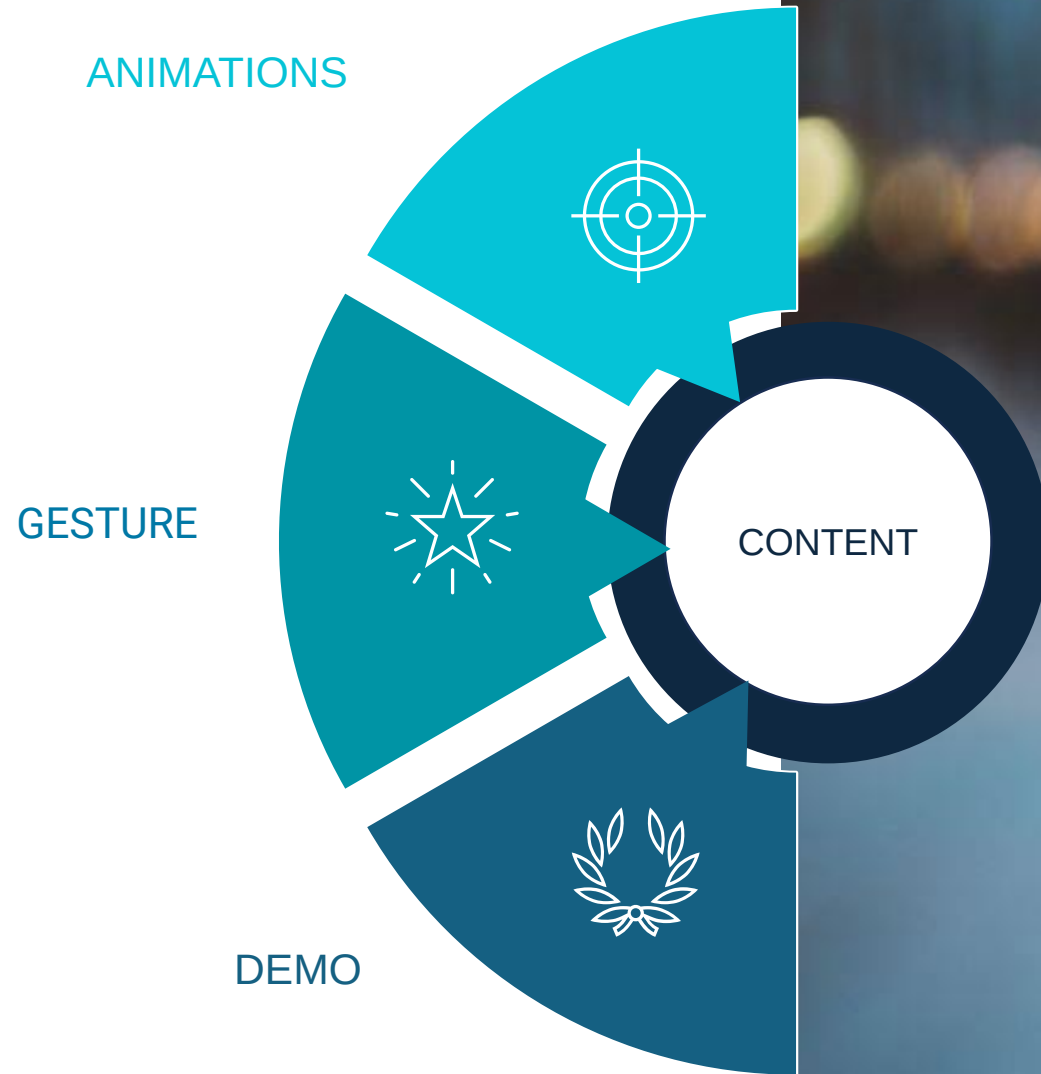




1

ANIMATIONS

Animated API

- Is designed to concisely express a wide variety of interesting animation and interaction patterns
- Animatable component types:
 - `Animated.View`
 - `Animated.Text`
 - `Animated.Image`
 - `Animated.ScrollView`
 - `Animated.FlatList`
 - `Animated.SectionList`

Animated Values

- Binding one or more style attributes of an animated component
 - `Animated.Value()` – single value
 - `Animated.ValueXY()` – vector

Configuring Animations

Animation type functions provides a particular animation curve that controls how your values animate from their **initial value** to the **final value**

TYPICAL – Animation types functions

- `Animated.timing()`
- `Animated.decay()`
- `Animated.spring()`

Animated.timing()

- Animates a value over time using easing functions
- **Animated.timing**(value, config)
 - animates a **value** along a timed easing curve
 - **config** is an object that may have the following options

Option	Description
duration	Length of animation (milliseconds). Default 500
easing	Easing function to define curve. Default is Easing.inOut(Easing.ease)
delay	Start the animation after delay (milliseconds). Default 0

Easing Module

Provides several predefined animations and helper:

Method	Description
back	provides a basic animation where the object goes slightly back before moving forward
bounce	provides a bouncing animation
ease	provides a basic inertial animation
in	runs an easing function forwards
out	makes any easing function symmetrical
inOut	runs an easing function backwards

Animated.decay()

- Starts with an initial velocity and gradually slows to a stop
- **Animated.decay(value, config)**
 - animates a value from an initial velocity to zero based on a decay coefficient
 - config is an object that may have the following options

Option	Description
friction	Controls "bounciness"/overshoot. Default 7
tension	Controls speed. Default 40
speed	Controls speed of the animation. Default 12
bounciness	Controls bounciness. Default 8

Animated.spring()

- Provides a basic spring physics model
- **Animated.spring**(value, config)
 - animates a value according to an analytical spring model based on damped harmonic oscillation
 - config is an object that may have the following options

Option	Description
friction	Controls "bounciness"/overshoot. Default 7
tension	Controls speed. Default 40
speed	Controls speed of the animation. Default 12
bounciness	Controls bounciness. Default 8

```
box: {  
  width: 100,  
  height: 100,  
  marginLeft: 10,  
  backgroundColor: '#0096c7'  
}
```



```
const Box = () => {
```

```
  const animatedValue = useRef(new Animated.Value(10)).current;
```

```
  const onStart = ()=>{
```

```
    Animated.timing(animatedValue,{  
      toValue: 200,  
      duration: 2000,  
      easing: Easing.linear,  
      useNativeDriver: false  
    }).start()
```

```
  }
```

```
  return (
```

```
    <View style={styles.container}>
```

```
      <Animated.View style={[styles.box,  
        {marginLeft: animatedValue} ]}/>
```

```
    </View>
```

```
  )
```

```
}
```

2:20



Start

Reset

Reanimated

- Is a powerful animation library built by **Software Mansion**
- Easily create smooth animations and interactions that run on the UI thread
- Animatable component types:
 - `Animated.View`
 - `Animated.ScrollView`
 - `Animated.FlatList`

useSharedValue

- Define shared values in your components
- Can be a value of any type
- Data stored in a shared value is automatically synchronized between the JavaScript thread and the UI thread

```
import { useSharedValue } from 'react-native-reanimated';  
  
const animatedValue = useSharedValue(10);  
const sv = useSharedValue({ x: 0, y: 0 });  
//modify value  
animatedValue.value = 200
```

useAnimatedStyle

- Create a styles object, similar to StyleSheet styles, which can be animated using shared values
- Styles defined using **useAnimatedStyle** have to be passed to **style** property of an Animated component
- Styles are automatically updated whenever an associated shared value or React state changes

```
const animatedStyles = useAnimatedStyle(() => ({
  marginLeft: withTiming(animatedValue.value,
    { duration: 2000,
      easing: Easing.bounce,
      reduceMotion: ReduceMotion.System})
})));
```

```
<Animated.View style={[styles.box, animatedStyles]} />
```


Configuration Animations

- `cancelAnimation`
- `withDecay`
- `withDelay`
- `withRepeat`
- `withSequence`
- `withSpring`
- `withTiming`

<https://docs.swmansion.com/react-native-reanimated/>

withTiming

➤ Arguments

- toValue (required)
- config (option)

Name	Type	Default	Description
duration	number	300	Length of the animation (in milliseconds).
easing	Easing	Easing.inOut(Easing.quad)	An easing function which defines the animation curve.
reduceMotion	ReduceMotion	ReduceMotion.System	A parameter that determines how the animation responds to the device's reduced motion accessibility setting.

```
box: {  
  width: 100,  
  height: 100,  
  marginLeft: 10,  
  backgroundColor: '#0096c7'  
}
```



```
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withTiming,
  Easing,
} from 'react-native-reanimated';
```

```
const Demo = () => {
  const animatedValue = useSharedValue(10);

  const animatedStyles = useAnimatedStyle(() => ({
    marginLeft: withTiming(animatedValue.value,
      { duration: 2000, easing: Easing.bounce}))
  ));

  const onStart = () => {
    animatedValue.value = 200;
  }

  return (
    <View style={styles.container}>
      <Animated.View style={[styles.box, animatedStyles]} />

    </View>
  );
}
```

2:20



Start

Reset

2

GESTURE

GESTURE RESPONDER SYSTEM

- Manages the lifecycle of gestures in your app
 - Scrolling
 - Sliding on a widget
 - Tapping

PANRESPONDER

- Handling several touches into a single gesture
- Used to recognize basic multi-touch gestures
- Provides a predictable wrapper of the responder handlers
- **Method:**

create(config: PanResponderCallbacks): PanResponderInstance

- 
- `onMove: (e, gestureState) => {...}`
 - `onMove: (e, gestureState) => {...}`
 - `onMove: (e, gestureState) => {...}`
 - `onStart: (e, gestureState) => {...}`
 - `onPan: (e, gestureState) => {...}`
 - `onPan: (e, gestureState) => {...}`
 - `onPan: (e, gestureState) => {...}`
 - `onPan: (e, gestureState) => {...}`
 - `onPanResponderMove: (e, gestureState) => {...}`
 - `onPanResponderTerminate: (e, gestureState) => {...}`
 - `onPanResponderTerminationRequest: (e, gestureState) => {...}`
 - `onShouldBlockNativeResponder: (e, gestureState) => {...}`

nativeEvent Object

- identifier - Unique numeric identifier assigned to the event
- locationX - Touch origin X coordinate inside touchable area (relative to the element)
- locationY - Touch origin Y coordinate inside touchable area (relative to the element)
- pageX - Touch origin X coordinate on the screen (relative to the root view)
- pageY - Touch origin Y coordinate on the screen (relative to the root view)
- Timestamp - Timestamp value when a PressEvent occurred. Value is represented in milliseconds

gestureState Object

- `stateID` - ID of the gestureState- persisted as long as there's at least one touch on screen
- `moveX` - the latest screen coordinates of the recently-moved touch
- `moveY` - the latest screen coordinates of the recently-moved touch
- `x0` - the screen coordinates of the responder grant
- `y0` - the screen coordinates of the responder grant
- `dx` - accumulated distance of the gesture since the touch started
- `dy` - accumulated distance of the gesture since the touch started
- `vx` - current velocity of the gesture
- `vy` - current velocity of the gesture
- `numberActiveTouches` - Number of touches currently on screen

```
import { View, Text, StatusBar, PanResponder, StyleSheet }
from 'react-native'
import React, { useRef } from 'react'
```

```
const PanResponderDemo = () => {
```

```
  const panResponder = useRef(PanResponder.create({
    onStartShouldSetPanResponder: () => true,
    onPanResponderMove: (evt, gestureState) => {},
    onPanResponderRelease: (evt, gesture) => {}
```

```
  })).current;
```

```
  return (
    <View style={styles.container} {...panResponder.panHandlers}>
      </View>
    )
  }
```

GestureHandler

- A library that provides an API for handling complex gestures
- Replace React Native's built in touch system called Gesture Responder System
- <https://docs.swmansion.com/react-native-gesture-handler/docs/>
- `npx expo install react-native-gesture-handler`
- `npx expo install react-native-reanimated`

GestureDetector

Component

- Responsible for creating and updating native gesture handlers based on the config of provided gesture
- Can recognize more than one gesture at the time

Gestures

Is the object that allows you to create and compose gestures

```
const tap = Gesture.Tap();  
return (  
  <GestureDetector gesture={tap}>  
    <Animated.View />  
  </GestureDetector>  
);
```

Base Gestures

- `Gesture.Tap()`
- `Gesture.Pan()`
- `Gesture.LongPress()`
- `Gesture.Fling()`
- `Gesture.Pinch()`
- `Gesture.Rotation()`
- `Gesture.ForceTouch()`

Composed Gestures

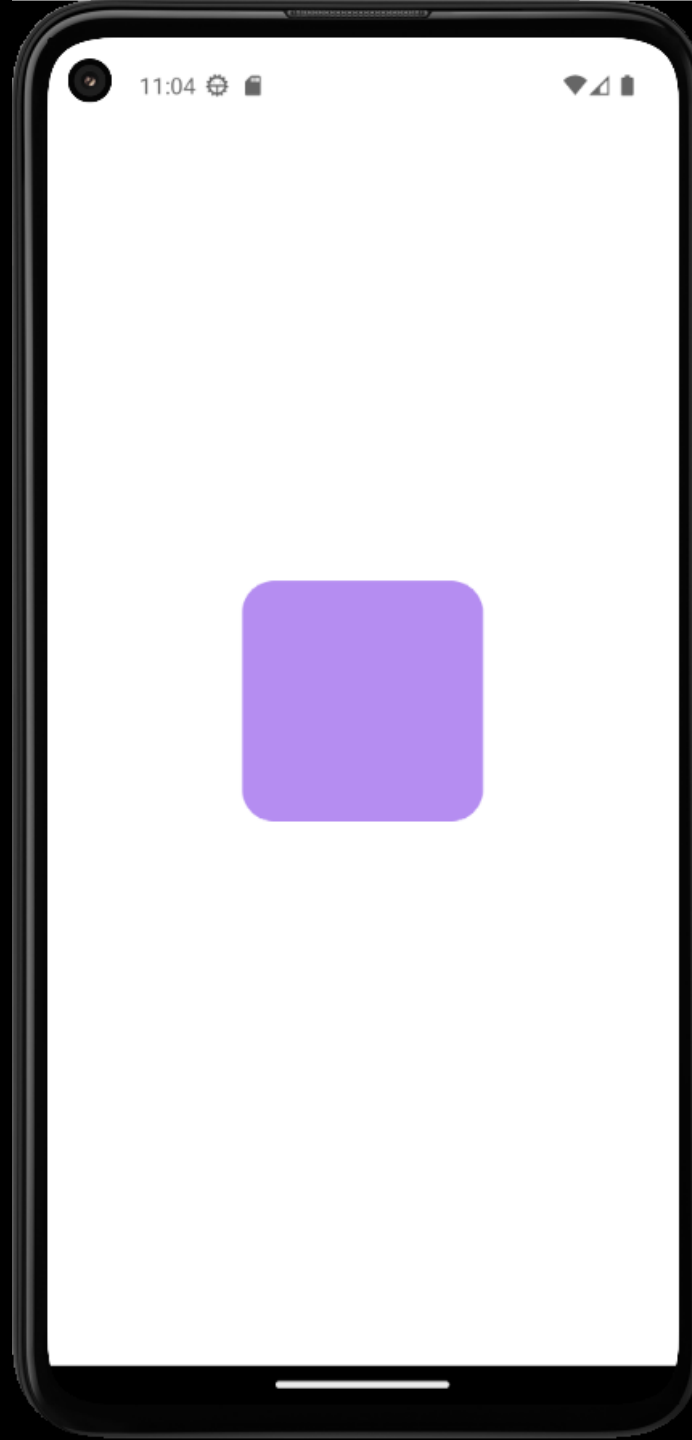
- Race
- Simultaneous
- Exclusive

```
const pan = Gesture.Pan();
const longPress = Gesture.LongPress();
const composed = Gesture.Race(pan, longPress);

return (
  <GestureDetector gesture={composed}>
    <Animated.View />
  </GestureDetector>
);
```


Common callbacks	Description
onBegin	called when given gesture handler starts receiving touches
onStart	called when the gesture is recognized by the handler and it transitions to the active state.
onEnd	called when the gesture that was recognized by the handler finishes.
onFinalize	called when the handler finalizes handling gesture - the gesture was recognized and has finished or it failed to recognize.
onTouchesDown	called every time a finger is placed on the screen
onTouchesMove	called every time a finger is moved on the screen
onTouchesUp	called every time a finger is lifted from the screen
onTouchesCancelled	called every time a finger stops being tracked, for example when the gesture finishes.

```
box: {  
  width: 150,  
  height: 150,  
  borderRadius: 20,  
  backgroundColor: '#b58df1',  
},
```



```
import React from 'react';
import { StyleSheet } from 'react-native';
import { Gesture, GestureDetector, GestureHandlerRootView, }
from 'react-native-gesture-handler';
```

```
const Demo = () => {
  const angle = useSharedValue(0);
  const startAngle = useSharedValue(0);
  const rotation = Gesture.Rotation()
    .onStart(() => {
      startAngle.value = angle.value;
    })
    .onUpdate((event) => {
      angle.value = startAngle.value + event.rotation;
    });
```

```
return (
  <GestureHandlerRootView style={styles.container}>
    <GestureDetector gesture={rotation}>
      <Animated.View />
    </GestureDetector>
  </GestureHandlerRootView>
);
```



```

const Demo = () => {

  const angle = useSharedValue(0);
  const startAngle = useSharedValue(0);
  const rotation = Gesture.Rotation()
    .onStart(() => {
      startAngle.value = angle.value;
    })
    .onUpdate((event) => {
      angle.value = startAngle.value + event.rotation;
    });

  const boxAnimatedStyles = useAnimatedStyle(() => ({
    transform: [{ rotate: `${angle.value}rad` }],
  }));

  return (
    <GestureHandlerRootView style={styles.container}>
      <GestureDetector gesture={rotation}>
        <Animated.View
          style={[styles.box, boxAnimatedStyles]} />

        </GestureDetector>
      </GestureHandlerRootView>
    );
}

```

